Multi-Agent Insurance system

Course Name : Agentic AI

Institution Name: Medicaps University

–Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	S. JEZREEL	EN22IT301085
2	KRITAGYA TIWARI	EN22IT301048
3	SADHANA YADAV	EN24CA5030146
4	CHANDRESH BHATI	EN24CA5030040
5	SHIVKUMAR SOLANKI	EN24CA5030157

Group Name: 02D12

Project Number:AAI-40

Industry Mentor Name:Mr. Ashwin sir

University Mentor Name:Mr. Ajeet singh Rajput

Academic Year:2025-26

Table of Contents

Chapter	Title	Page
Chapter 1	Introduction	
	1.1 Problem Statement	3
	1.2 Project Objective	3
	1.3 Scope of the Project	4
Chapter 2	Proposed Solution	
	2.1 Key Features	5
	2.2 Overall Architecture / Workflow	6
	2.3 Tool and Technologies	12
Chapter 3	Result and Output	
	3.1 Screenshots / Outputs	15
	3.2 Models	18
	3.3 Key outcomes	20
Chapter 4	Conclusion	
	4.1 Conclusion	21
	4.2 Future and Scope	23

Chapter 1: Problem Statement & Objectives

1.1 Problem Statement

The Multi Agent Insurance System is designed to implement a collaborative architecture of specialized AI agents that work together to process insurance-related queries. The system consists of a Researcher Agent responsible for retrieving relevant information such as policy coverage details, and a Writer Agent that transforms this information into structured and user-friendly responses. An Agent Orchestrator manages the interaction and hand-off between these agents, ensuring smooth workflow execution. The system integrates with the Groq API for natural language processing. This project demonstrates efficient inter-agent communication, dynamic state management, and the ability to generate high-quality, structured outputs through coordinated multi-agent workflows. Additionally, the system emphasizes modularity and scalability, allowing future integration of additional agents and enhanced functionalities. It highlights the practical application of AI-driven automation in simplifying complex insurance information for better user understanding.

1.2 Project Objective

The primary objective of the Multi Agent Insurance System is to design and implement an intelligent, collaborative framework that utilizes multiple specialized AI agents to process and generate structured insurance-related information. The system aims to improve the accuracy, clarity, and usability of insurance content by dividing tasks between a Researcher Agent and a Writer Agent. It focuses on efficient inter-agent communication, seamless task hand-off, and optimized use of LLM capabilities through the Groq API. Additionally, the project aims to demonstrate scalability, modular design, and the practical application of multi-agent systems in solving real-world problems related to information processing and automation.

1.3 Scope of the Project :

The scope of the Multi Agent Insurance System is to design and implement a collaborative architecture of specialized AI agents that efficiently process insurance-related queries. The system focuses on enabling users to input queries related to insurance topics such as policy coverage, benefits, and general information. The system operates in a real-time environment and integrates with the Groq API for natural language understanding and content generation.

The current scope is limited to handling text-based insurance queries and does

not include persistent data storage, user authentication, or advanced analytics. However, the system provides a strong foundation for future enhancements such

as database integration, support for multimodal inputs, and expansion into other domains beyond insurance. Overall, the project demonstrates the practical application of multi-agent systems in automating information processing and improving user understanding through structured and intelligent outputs.

Chapter 2: Proposed Solution

2.1 Key Features:

The Multi Agent Insurance System is built around a simple idea: break a complex task into smaller, focused jobs and assign each one to a dedicated AI agent. This makes the system easier to manage, more accurate, and straightforward to extend in the future.

Specialised Agents

The system has two agents, each with a clear job. The Researcher Agent collects accurate, India-specific insurance information – things like policy coverage, premium ranges, IRDAI rules, and tax benefits. The Writer Agent then takes that raw information and turns it into a clean, well-structured response that is easy for any user to read and understand.

Agent Orchestrator with Handoff Protocol

The Agent Orchestrator – built using LangChain – acts as the traffic controller for the entire pipeline. It decides when the Researcher has done enough work, validates the collected data through a Handoff Gate, and only then passes the research to the Writer. This ensures no incomplete information ever reaches the final response.

RAG Pipeline with ChromaDB

Before calling the LLM, the Researcher Agent searches a local vector database (ChromaDB) that contains 22 curated insurance knowledge chunks specific to the Indian market. This is called Retrieval-Augmented Generation (RAG). By grounding the LLM response in real, pre-verified data, the system produces answers that are far more accurate and relevant than a plain LLM call alone.

Groq LLM Integration

Both agents use the Groq API running the Llama 3.3 70B model for natural language understanding and generation. Groq is chosen specifically for its speed – it delivers responses in under 2 seconds – which keeps the overall pipeline fast

even with two separate LLM calls.

Multimodal AI Studio

Alongside the agent pipeline, the system includes an AI Studio where users can also generate visual infographics for any insurance topic. Image generation is handled by Google's Gemini Flash Image API, with HuggingFace FLUX.1-Dev as a fallback – the same model used by Raphael AI.

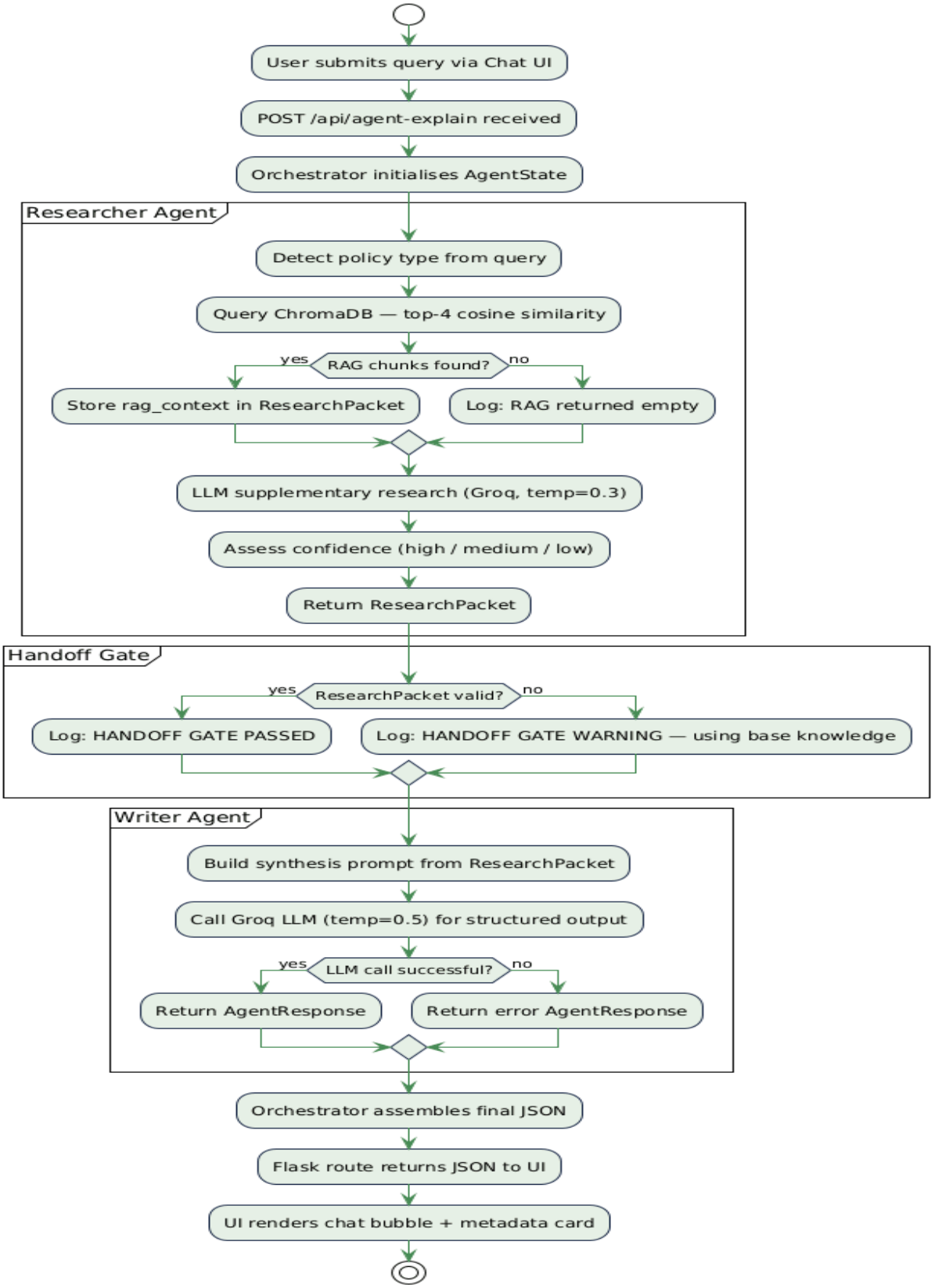
Modular and Scalable Architecture

Every component – the orchestrator, each agent, the knowledge base, the Flask routes – is written as a separate, independent module. Adding a new agent (for example, a Validator Agent) requires no changes to existing code. This keeps the system clean and easy to maintain.

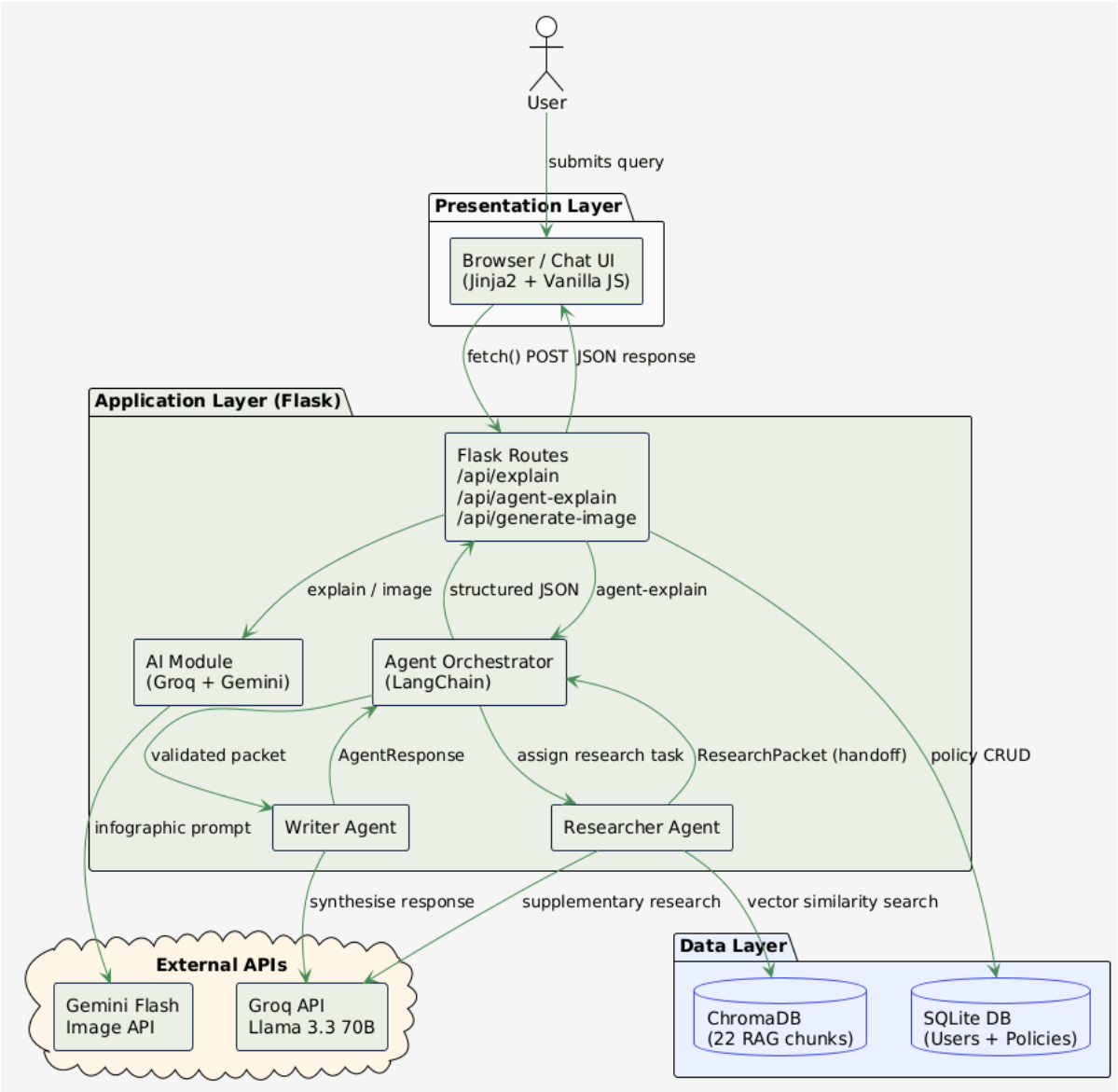
Privacy-First, Stateless Processing

No user queries or agent outputs are stored permanently. Each request is handled independently in memory and cleared once the response is sent. This keeps the system lightweight and ensures user privacy by design.

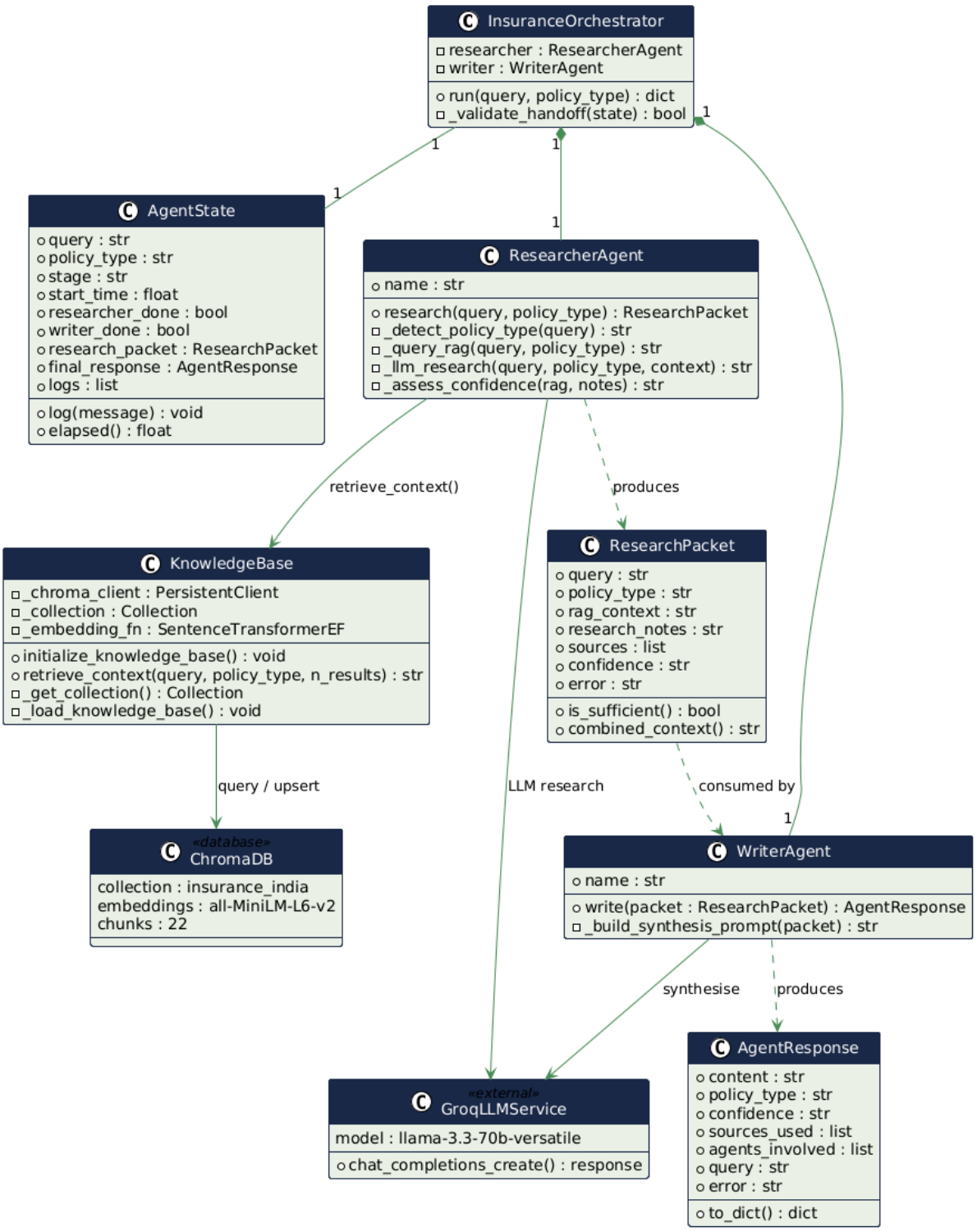
2.2 Overall Architecture / Workflow:


User submits query via Chat UI
POST /api/agent-explain received
Orchestrator initialises AgentState
Researcher Agent
Detect policy type from query
Query ChromaDB — top-4 cosine similarity
yes
RAG chunks found?
no
Store rag_context in ResearchPacket
Log: RAG returned empty
LLM supplementary research (Groq, temp=0.3)
Assess confidence (high / medium / low)
Return ResearchPacket
Handoff Gate
yes
ResearchPacket valid?
no
Log: HANDOFF GATE PASSED
Log: HANDOFF GATE WARNING — using base knowledge
Writer Agent
Build synthesis prompt from ResearchPacket
Call Groq LLM (temp=0.5) for structured output
yes
LLM call successful?
no
Return AgentResponse
Return error AgentResponse
Orchestrator assembles final JSON
Flask route returns JSON to UI
UI renders chat bubble + metadata card

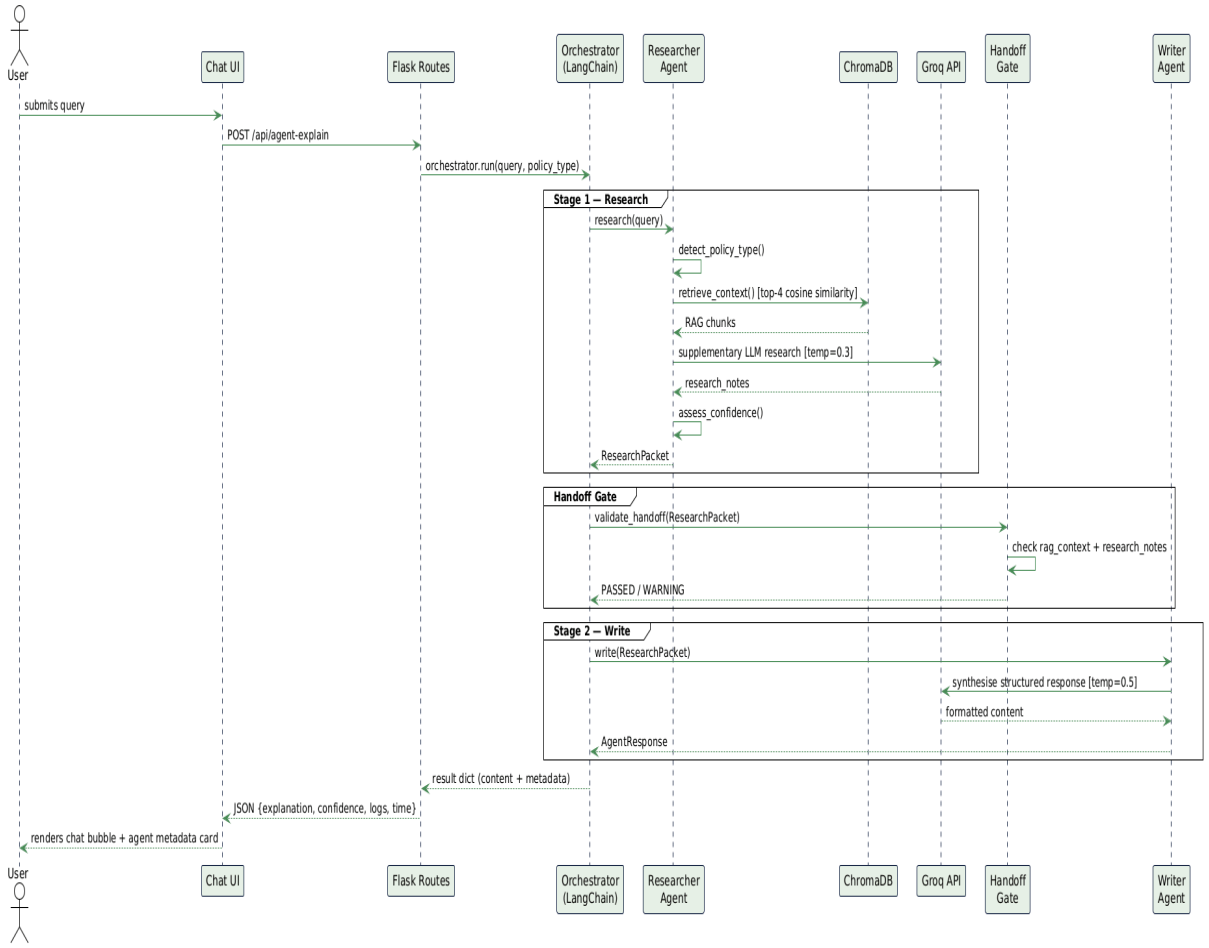
High level diagram:



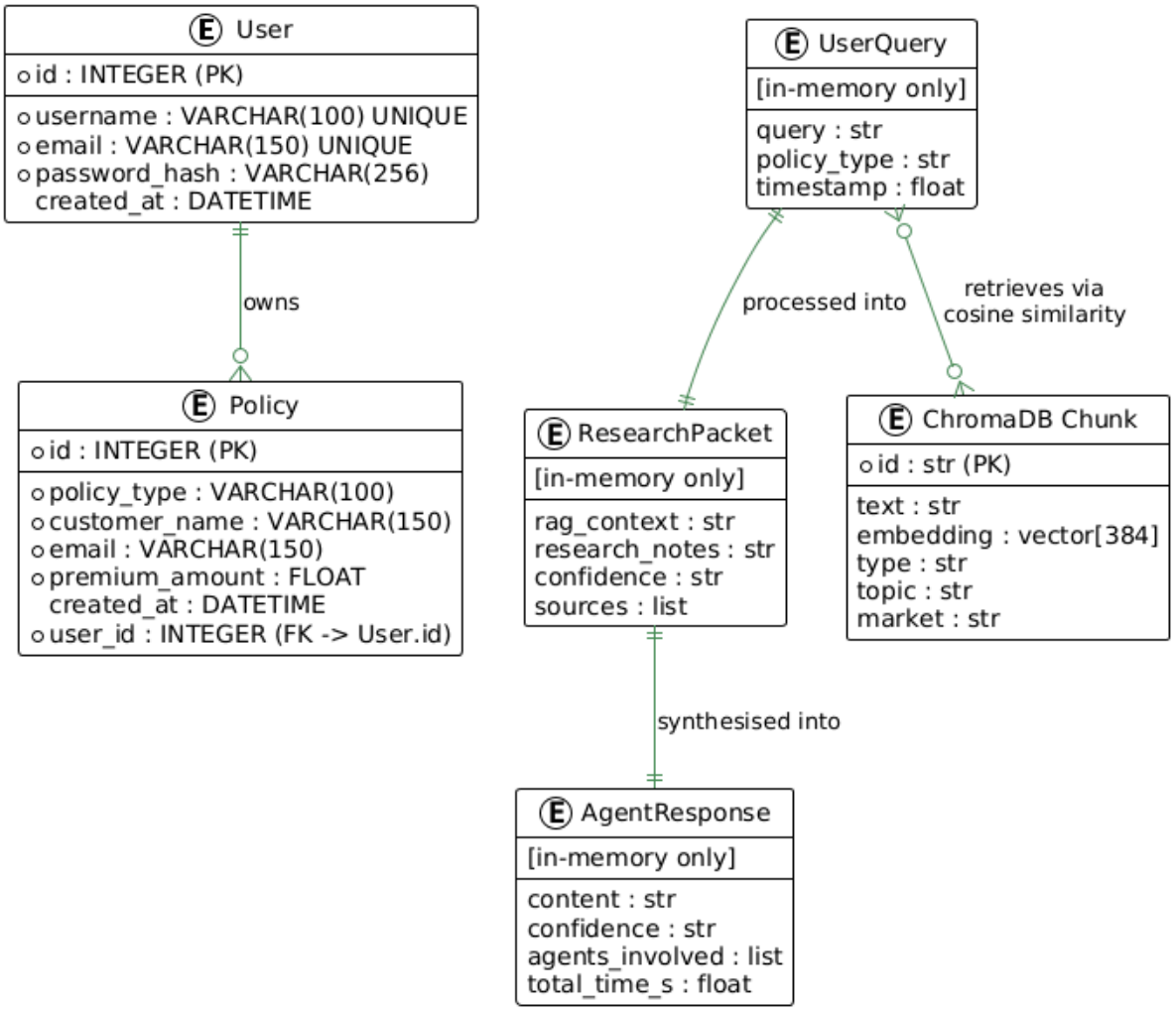
Class diagram:



Sequence Diagram :



ER Diagram :


User
id : INTEGER (PK)
username : VARCHAR(100) UNIQUE
email : VARCHAR(150) UNIQUE
password_hash : VARCHAR(256)
created_at : DATETIME
owns
Policy
id : INTEGER (PK)
policy_type : VARCHAR(100)
customer_name : VARCHAR(150)
email : VARCHAR(150)
premium_amount : FLOAT
created_at : DATETIME
user_id : INTEGER (FK -> User.id)
UserQuery
[in-memory only]
query : str
policy_type : str
timestamp : float
processed into
retrieves via
cosine similarity
ResearchPacket
[in-memory only]
rag_context : str
research_notes : str
confidence : str
sources : list
ChromaDB Chunk
id : str (PK)
text : str
embedding : vector[384]
type : str
topic : str
market : str
synthesised into
AgentResponse
[in-memory only]
content : str
confidence : str
agents_involved : list
total_time_s : float

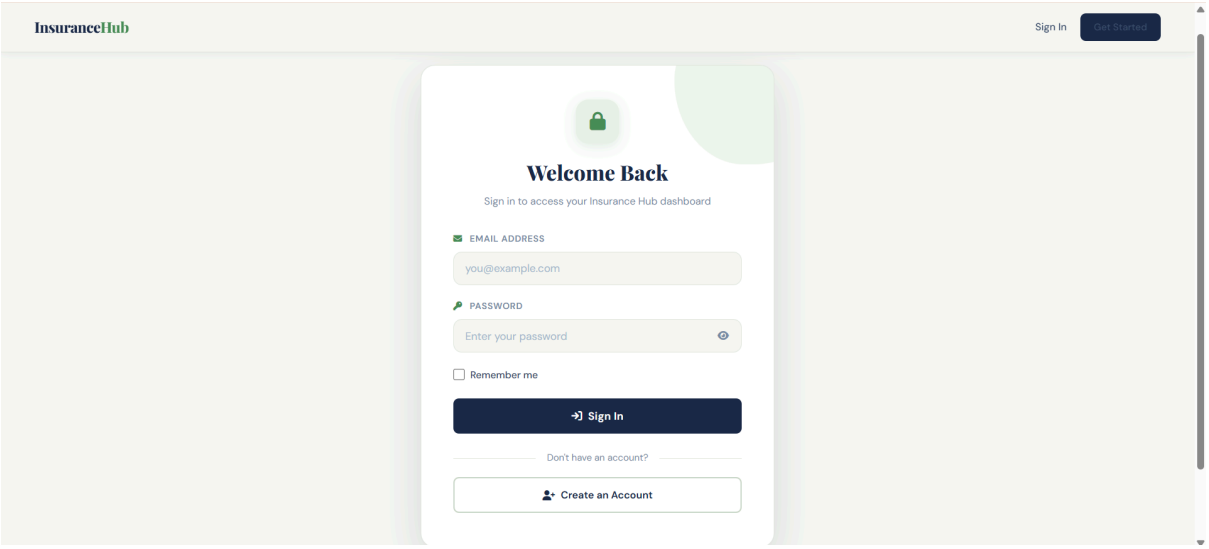
2.3 Tools & Technologies :

Category	Tools / Technology	Purpose in Project
Web Framework and programming language	Python Flask	Handles routing, templates, and REST API endpoints.
Database (ORM)	SQLite + SQLAlchemy	Stores user accounts and insurance policy records.
Authentication	Flask-Login + Werkzeug	Session management and password hashing (PBKDF2-SHA256)
LLM Inference	Groq API — Llama 3.3 70B	Powers both Researcher and Writer agents for text generation
Agent Orchestration	LangChain	Manages agent workflow, task assignment, and state management
Vector Database	ChromaDB	Stores and retrieves 22 India-specific RAG knowledge chunks
Embeddings	sentence-transformers (MiniLM)	Converts knowledge chunks to vector embeddings for similarity search
Image Generation	Gemini Flash Image API	Primary text-to-image generator for insurance infographics

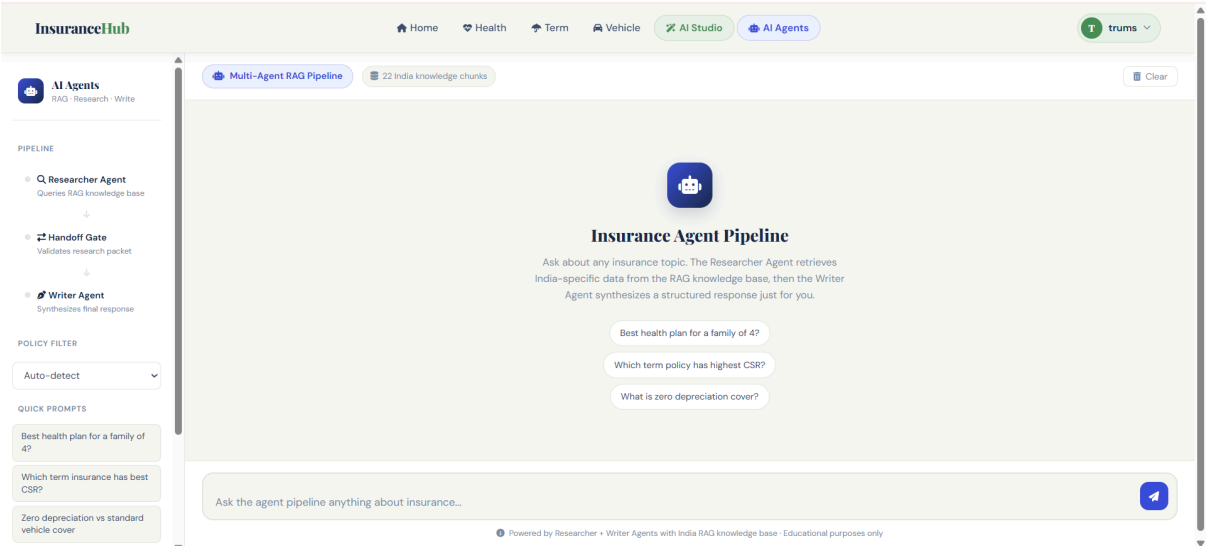
Image Fallback	HuggingFace FLUX.1-Dev	Backup image generator (same model as Raphael AI)
Frontend	Jinja2 + Vanilla JS + CSS	Chat-style UI with async fetch() calls and real-time progress
Icons / Fonts	Font Awesome 6 + Google Fonts	UI icons and typography (Playfair Display, DM Sans)
Containerisation	Docker	Packages the app and its dependencies into a portable image.
Orchestration	Kubernetes (kubeadm)	Deploys and manages containers on the cloud cluster
API Client	Requests (Python)	Makes HTTP calls to Groq, Gemini, and HuggingFace APIs
Image Processing	Pillow (PIL)	Converts and encodes generated images to base64 data URLs

Chapter 3: Result and Output

3.1 Screenshots / Outputs :

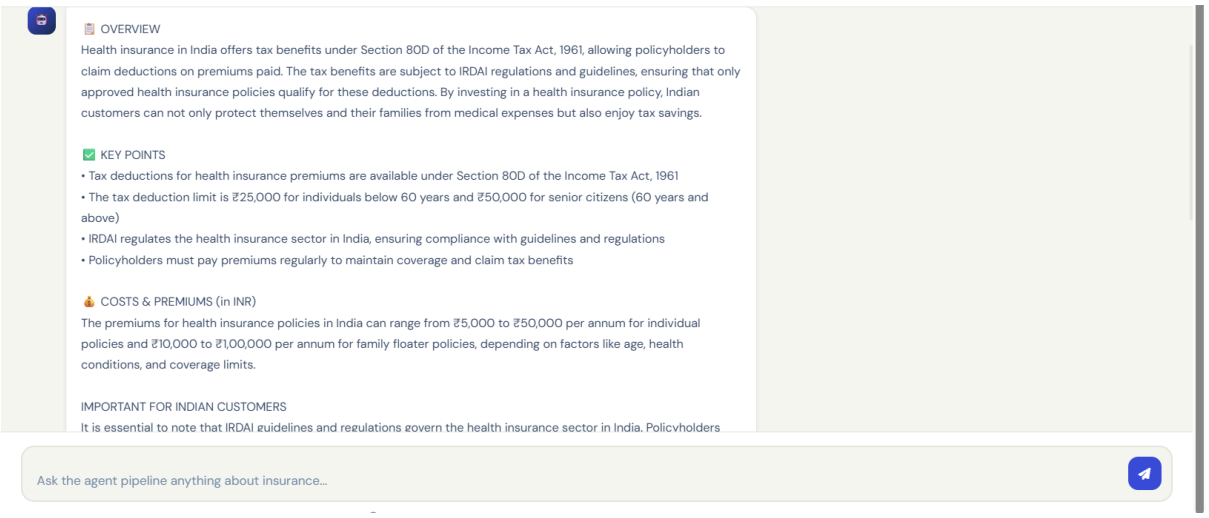


Login/Authentication page



Insurance Agent webpage

Chat section and workflow



Chat response

```
127.0.0.1 - - [21/Apr/2026 18:17:48] "GET /static/js/script.js HTTP/1.1" 304 -
[Orchestrator] Pipeline started for query: 'Tax benefits of health insurance India'
[Orchestrator] Stage 1: Invoking ResearcherAgent...
[ResearcherAgent] Starting research: 'Tax benefits of health insurance India...'
[ResearcherAgent] Policy type detected: Health Insurance
[ResearcherAgent] Querying RAG knowledge base...
[RAG] Init error: [WinError 1114] A dynamic link library (DLL) initialization routine failed. Error loading "G:\student_st
reamlit\Insurance_app\venv\Lib\site-packages\torch\lib\c10.dll" or one of its dependencies.
[ResearcherAgent] RAG retrieved 0 relevant chunks.
[ResearcherAgent] Running LLM supplementary research...
[ResearcherAgent] Research confidence: high
[ResearcherAgent] Research complete. Handing off to Writer Agent.
[Orchestrator] ResearcherAgent complete. Policy type: Health Insurance. Confidence: high.
[Orchestrator] HANDOFF GATE: PASSED — RAG chunks: no, Research notes: yes, Confidence: high
[Orchestrator] Stage 2: Invoking WriterAgent (handoff received)...
[WriterAgent] Received handoff from ResearcherAgent.
[WriterAgent] Synthesizing response for: 'Tax benefits of health insurance India...'
[WriterAgent] Research confidence: high
[WriterAgent] Response synthesized successfully (2638 chars).
[Orchestrator] WriterAgent complete. Response length: 2638 chars.
[Orchestrator] Pipeline complete in 48.89s.
127.0.0.1 - - [21/Apr/2026 18:19:09] "POST /api/agent-explain HTTP/1.1" 200 -
```

3.2 Models :

The proposed system integrates multiple Artificial Intelligence models to enable accurate, fast, and context-aware insurance assistance. These models are used across different stages of the pipeline, including information retrieval, natural language generation, and image synthesis.

Large Language Model (LLM):
The system utilizes the Llama 3.3 70B model, accessed via the Groq API, as the core language model for both the Researcher and Writer agents. This model is responsible for understanding user queries, generating context-aware responses, and transforming structured research into user-friendly explanations. Groq's high-speed inference ensures low latency, allowing the system to deliver responses in near real-time.

Embedding Model:
For semantic search within the Retrieval-Augmented Generation (RAG) pipeline, the system employs a MiniLM-based model from the Sentence-Transformers library. This model converts preprocessed insurance knowledge chunks into dense vector representations. These embeddings are stored in ChromaDB and enable efficient similarity-based retrieval of relevant information before invoking the LLM.

Image Generation Models:
To support multimodal capabilities, the system integrates text-to-image generation models. The primary model used is Gemini Flash Image API, which generates high-quality insurance-related infographics based on user prompts. In case of fallback scenarios, the FLUX.1-Dev model from HuggingFace is used to ensure uninterrupted image generation functionality.

Model Integration Strategy:
These models are orchestrated using LangChain, which manages the interaction between the embedding model, vector database, and LLM agents. The embedding model first retrieves relevant context, which is then passed to the LLM for grounded response generation. Image models operate in parallel within the AI Studio module to provide visual outputs.

This multi-model architecture enhances the system's accuracy, scalability, and user experience by combining the strengths of retrieval-based and generative AI approaches.

3.3 Key Outcomes:

The Multi Agent Insurance System successfully delivers a structured and intelligent approach to processing insurance-related queries using a collaborative multi-agent architecture. One of the key outcomes of the project is the effective implementation of specialized agents, where the Researcher Agent retrieves relevant information and the Writer Agent transforms it into clear, well-organized outputs. This division of responsibilities improves both accuracy and efficiency in handling complex queries.

Another important outcome is the successful orchestration of agent interactions through a centralized Agent Orchestrator. This ensures seamless communication, proper task hand-off, and smooth workflow execution. The integration of the Groq API enables the system to generate high-quality, context-aware responses using natural language processing capabilities.

The project also demonstrates real-time data processing without reliance on persistent storage, making the system lightweight and flexible. The use of shared state management allows efficient data exchange between agents, ensuring consistency in outputs. Additionally, the modular design of the system supports scalability and future enhancements.

Overall, the system achieves its objective of providing accurate, structured, and user-friendly insurance information, showcasing the practical application of multi-agent AI systems in real-world scenarios and improving the overall efficiency of information processing

Additionally, the system achieves a high level of flexibility and scalability due to its modular architecture. The design allows easy integration of new agents, improved workflows, or additional functionalities without disrupting existing components. This outcome ensures that the system is future-ready and adaptable to evolving requirements. It also demonstrates how multi-agent systems can be effectively used to build intelligent, scalable, and efficient solutions for automated information processing.

Chapter 4: CONCLUSION

4.1 Conclusion :

The Multi Agent Insurance System successfully demonstrates the design and implementation of a collaborative AI-based architecture for processing insurance-related queries. By utilizing specialized agents such as the Researcher Agent and Writer Agent, the system effectively divides complex tasks into manageable components, resulting in accurate and structured outputs. The use of an Agent Orchestrator ensures smooth coordination and efficient communication between agents, enabling a seamless workflow.

The integration of the Groq API enhances the system's ability to understand and generate meaningful responses using advanced Large Language Model capabilities. The system operates in a real-time and stateless manner, ensuring flexibility, efficiency, and ease of scalability. Its modular design allows for future enhancements without affecting the core architecture.

Overall, the project highlights the practical application of multi-agent systems in solving real-world problems, particularly in simplifying complex insurance information for users. It demonstrates how AI-driven automation can improve user experience, accuracy, and efficiency in information processing. The system provides a strong foundation for future advancements, making it a scalable and innovative solution in the domain of intelligent insurance assistance.

4.2 Future Scope and Enhancement

The Multi Agent Insurance System has significant potential for future enhancements and expansion. One major improvement could be the integration of additional specialized agents, such as a Validation Agent or Recommendation Agent, to further enhance the accuracy and usefulness of responses. The system can also be extended to support multimodal inputs, including voice or document-based queries, making it more interactive and user-friendly.

Another important enhancement is the integration of a database to store user queries, responses, and analytics data. This would enable personalized recommendations and better decision-making based on user history. The system can also be improved by incorporating advanced orchestration techniques using

frameworks like LangChain or CrewAI to manage more complex workflows and agent collaboration.

Performance optimization through caching mechanisms and asynchronous processing can reduce response time and improve scalability. Additionally, deploying the system on cloud platforms with containerization can enhance reliability and availability. Security features such as role-based access control and secure API handling can also be implemented.

Further enhancements may include integrating domain-specific knowledge bases to improve response accuracy and reduce dependency on external APIs. The system can also support multilingual capabilities to serve a wider range of users. Incorporating feedback mechanisms will allow continuous learning and improvement of agent responses. Overall, the system can evolve into a highly intelligent, scalable, and production-ready solution for real-world insurance assistance.